



Software Requirements Specification Version 1.0

Project Name: BudgetBasics
Theme: NextGen BudgetBee
Category: Web Innovation Unleashed

Table of Contents

1.1	Background and Necessity for the Website	3
1.2	Proposed Solution	4
1.3	Purpose of the Document	5
1.4	Scope of Project	7
1.5	Constraints	7
1.6	Functional Requirements	7
1.7	Non-Functional Requirements.....	13
1.8	Interface Requirements	14
1.9	Project Deliverables.....	15

1.1 Background and Necessity for the Website

Managing personal finances is an important life skill. Many students start handling allowances, scholarships, internship income, or part-time earnings without a simple way to plan and track their money.

Small daily expenses can quickly reduce the money available for important requirements, learning resources, transportation, and savings.

A student-friendly Website can make financial concepts easier to understand by using clear explanations, relatable examples, visual infographics, and simple interactive tools.

'BudgetBasics' is proposed as a solution that will help learners understand where money comes from, how it is spent, and how small planning decisions can build better money habits. It can aid them to:

- Understand income, expenses, requirements, wants, and savings
- Learn how to divide a monthly allowance or income into useful categories
- Notice overspending, impulse buying, unused subscriptions, and other common money mistakes
- Set simple savings goals and estimate how long they will take
- Use visual examples and guided questions to build responsible money habits

The Website will support students to plan better, avoid common spending problems, and improve financial discipline.



1.2 Proposed Solution

You will design and build a responsive educational Website named **BudgetBasics**, focused on helping students learn personal budgeting fundamentals through simple guides, infographics, examples, and interactive financial awareness content.

BudgetBasics will provide education and awareness about budgeting, saving, avoiding common money mistakes, and developing responsible spending habits. The Website will be a Single Page Application (SPA) built using technologies such as HTML, CSS, ReactJS/Angular/Bootstrap, and JavaScript with no backend database or server-side storage. Key features for users:

- Learn budgeting basics through beginner-friendly guides
- View infographics explaining student spending categories
- Understand saving methods and goal planning
- Identify and avoid common money mistakes
- Use an AI-powered Chatbot for basic financial queries
- Receive AI-generated tips based on common student spending habits



The Website will include visual learning sections, budgeting examples, static calculators or estimators, tip cards, downloadable-style checklists, and student-friendly awareness content presented through attractive layouts.

Scope consistency

BudgetBasics is an educational Website. It may demonstrate budget calculations, but it must not claim to provide banking services, real financial transactions, or permanently stored personal financial records.

1.3 Purpose of the Document

The purpose of this document is to describe **'BudgetBasics'** in detail. It explains the Website's educational features, user interface, technical limits, functional requirements, and project deliverables.

This document:

- Defines system requirements
- Describes features and functionality
- Identifies user roles
- Specifies constraints
- Defines user interface



1.3.1 Who Can Use This Document?

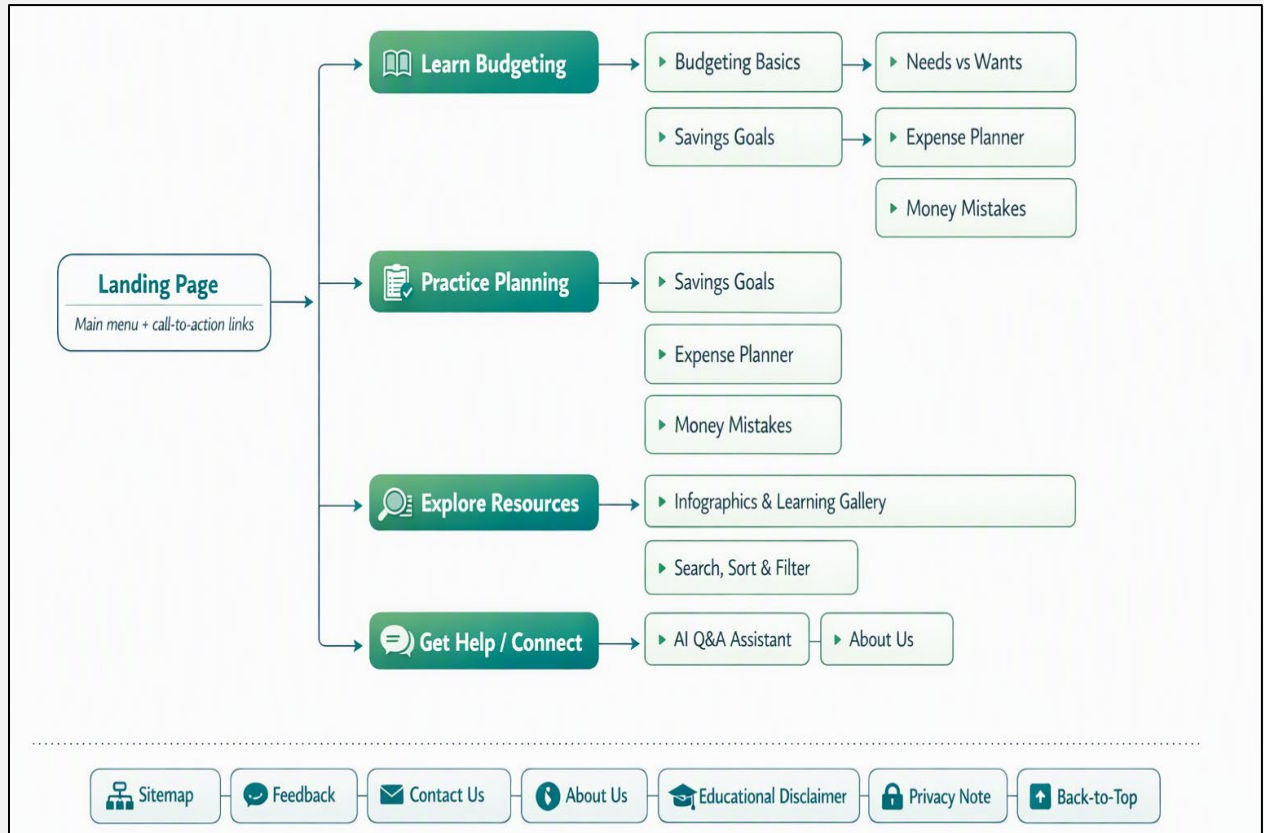
This document can be used by students, trainers, evaluators, and project reviewers to understand the expected scope, features, design constraints, and deliverables of the **'BudgetBasics'** Website. It also helps developers confirm which modules must be implemented and which features are intentionally excluded from the project.

This document can be used by	Stakeholders
	Project Managers
	Developers
	Testers
	Designers
	Students implementing the project

Primary Audience	Students, college learners, and beginners in personal finance
Implementation	HTML5, CSS3, JavaScript, (optional) Bootstrap/Angular/React, and JSON/TXT content files and AI plugins as applicable, UI and UX frameworks as applicable
Data Model	Pre-populated via tables/JSON content

Sample Site Map

BudgetBasics Information Architecture and Sitemap Flow



Data note

Values displayed in examples, calculators, tips, and the AI Chatbot assistant may be hard-coded or retrieved from pre-populated JSON/TXT files. Form submissions do not require to be saved to a server.

1.4 Scope of Project

The system will be developed as a responsive Website for financial education and student budgeting awareness.

Website will Allow	Website must Work on
• Budgeting fundamentals • Saving awareness and goal planning • Common money mistake prevention • Infographic-based learning • AI-assisted basic finance Chatbot and tips	• Desktop • Tablet • Mobile Devices

1.5 Constraints

The portal will operate as a responsive Website without backend services, server-side database integration, login storage, or transaction processing.

Data may be Retrieved Using
• Static and responsive HTML pages • JavaScript files for interactions • Optional JSON/TXT files for pre-defined tips, FAQs, and sample spending categories

1.6 Functional Requirements

Following are the functional requirements of the Website:

- Display the **BudgetBasics** logo, project tagline, and a clear welcome banner.
- Provide navigation to Budgeting Basics, Needs vs Wants, 50-30-20 Rule, Savings Goals, Expense Planner, Money Mistakes, Infographics, AI Chatbot, Feedback, and Contact Us.
- Show featured budgeting tips, quick facts, and clear calls to action.
- Include a visitor counter and real-time date/time display.
- Provide a visible link to the sitemap.

1. Budgeting Basics Module

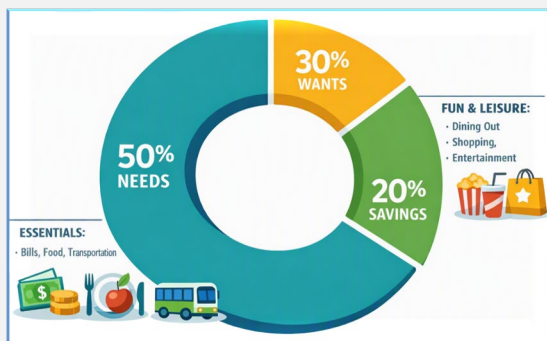
- Display information on income, fixed expenses, variable expenses, requirements, wants, and savings.
- Use cards, examples, tables, or illustrations to present each concept.
- Display a sample student monthly budget using clearly labeled sample currency values.
- Provide a short knowledge check or interactive question.

2. Needs vs. Wants Module

- Display examples of essential and optional spending categories.
- Allow users to classify sample items as a need or a want.
- Provide feedback explaining the selected answer.
- Show a visual decision guide that helps users delay non-essential purchases.



3. 50-30-20 Budget Module



- Explain the suggested split of 50% for needs, 30% for wants, and 20% for savings.
- Display Calculator outputs for the formula used, validate blank or invalid inputs, show results with clear labels, and include a short educational note explaining that the result is an estimate for

learning purposes only.

- Accept a sample monthly income value and calculate the three suggested amounts.
- Display the result using labels, progress bars, or a chart.
- State clearly that the split is an educational guideline and may be adjusted.

4. Savings Goals

- Allow users to enter a goal name, target amount, current savings, and expected monthly contribution.
- Calculate the remaining amount and estimated months required to achieve the goal.
- Display a progress bar and an encouraging savings tip.
- Validate empty, negative, or non-numeric values.



5. Expense Planner Demonstration

- Allow users to add sample expense entries containing date, category, description, and amount.



- Display entered items into a temporary on-screen table.
- Calculate total planned expenses and remaining sample balance.

- Allow users to edit or remove entries during the current

session.

- Provide category options such as Food, Transport, Education, Entertainment, Shopping, Utilities, and Miscellaneous.

6. Money Mistakes

- Explain common mistakes such as impulse buying, ignoring small expenses, late payments, unused subscriptions, and spending without a plan.
- Present each mistake with a realistic student scenario and a corrective action.
- Allow users to expand or collapse tips using interactive cards or accordions.

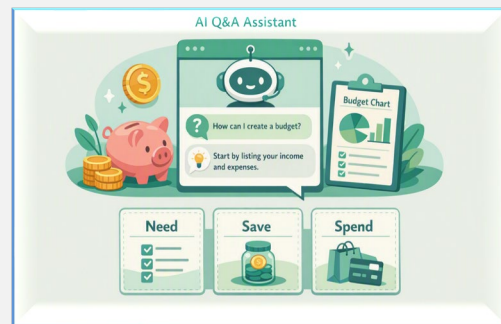


7. Infographics and Learning Gallery

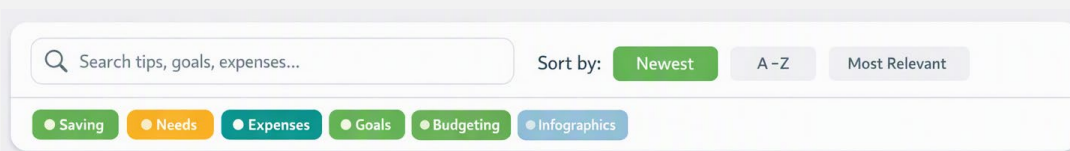
- Display original visual content such as Needs vs. Wants, the 50-30-20 split, a monthly budget cycle, and saving challenges.
- Provide readable captions and suitable alternative text for meaningful images.
- Allow users to filter the gallery by topic where applicable.

8. AI Chatbot Assistant

- Provide a question input area and an **Ask** button.
- Respond to common questions using pre-defined JSON responses, keyword matching, or rule-based JavaScript logic.
- Display suggested prompts such as 'What is a need?', 'How much should I save?', and 'How do I avoid overspending?'.
- Show a safe fallback message when a question is outside the available educational topics.
- Display a disclaimer that the feature provides basic educational information, not professional financial advice.



9. Search, Sort, and Filter



- Allow users to search learning content by keywords such as saving, needs, expenses, or goals.
- Allow filtering of tips, examples, or infographics by topic.
- Display a clear message when no matching content is found.

10. About Us, Feedback, and Contact Us

- Display information about the purpose and creators of the Website.
- Feedback and contact forms should perform client-side validation only. After validation, the Website should display a confirmation

message, but it should not submit, store, or transmit personal information unless a separate backend requirement is formally added.

- Provide a validated feedback form with name, email, rating, and comments.
- Display contact information such as email address, phone number, and social media links.
- Show a confirmation message after successful client-side form validation

11. Additional Website Features

- Responsive navigation menu with active, hover, and focus states.
- Optional dark mode with readable contrast.
- Animated learning cards and smooth section transitions.
- Financial quotes or tips ticker.
- Back-to-top control and clear footer navigation.
- Keyboard-accessible controls and visible focus indicators

Use Google Lighthouse, free tool, to test and validate your Website's performance, accessibility, and SEO.

Important Note Regarding AI Usage:

You are encouraged to use AI-powered tools (such as AI-assisted Website builders, UI/UX design tools, code assistants, and image-generation tools) to enhance productivity and creativity. However, AI should be used as a supporting aid rather than a substitute for your own design, development, and problem-solving skills.

Do NOT rely on full boilerplate or completely ready-made Website templates for your project, as this would adversely affect your evaluation. Your Website's design, structure, and implementation should primarily reflect your own skills and understanding.

Do NOT submit AI-generated code or content without meaningful modification and understanding. AI-generated suggestions may be used for guidance, learning, debugging, or improving productivity, but the final solution should demonstrate your own effort, logic, and implementation.

AI-generated images and graphics are permitted for visual elements of your Website.

Acknowledge the AI tool(s) used (for example, Canva AI, Figma AI, Uizard, or similar) in your project documentation or submission. During evaluation, judges may ask participants to explain their design decisions, implementation approach, and code. You are therefore expected to understand and be able to justify all aspects of their submitted work.

1.7 Non-Functional Requirements

Following quality requirements must be fulfilled by the Website:

Requirement	Description
Safe to Use	The Website should not trigger malicious, unexpected, or unnecessary downloads. External links should be clearly identified.
Accessibility	Fonts, controls, images, navigation, and form elements should be clear, legible, keyboard accessible, and supported by suitable labels or alternative text.
User-Friendliness	The Website should use simple language, clear menus, consistent controls, and understandable feedback messages.
Operability	All required modules, links, calculations, forms, and interactive controls should operate reliably.
Performance	Pages and sections should load quickly, animations should remain smooth, and user actions should receive prompt feedback.
Reliability	The Website should provide stable results for valid inputs and handle invalid values without crashing or producing misleading output.
Scalability	The content structure should support additional tips, questions, examples, or learning modules without redesigning the entire Website.
Availability	The Website should remain available whenever it is hosted, with minimal dependency on unavailable external services.
Compatibility	The Website should work on recent versions of Chrome, Firefox, Edge, and Safari and adapt to desktop, tablet, and mobile devices.
Maintainability	HTML, CSS, JavaScript, and content files should be organized, commented, and modular enough to support future updates.
Privacy	The Website should avoid collecting sensitive financial or banking data. Any temporary demonstration data should be removable and should not be transmitted without a clear requirement.
Accuracy of Educational Output	Calculators must use correct formulas, clearly label assumptions, and display educational estimates in an understandable format.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS. Once they are complete, you can use your own creativity and imagination to add more features if required.

1.8 Interface Requirements

1.8.1 Hardware

Component	Minimum Requirement
Processor	Intel Core i5/i7 Processor or higher
Memory	8 GB RAM or higher
Display	Color SVGA
Storage	500 GB Hard Disk space
Input Devices	Mouse and keyboard
Network	Internet connection for optional external resources and browser testing

1.8.2 Software

Software Area	Recommended Tools / Technologies
IDE	Visual Studio Code, Coffee Cup Free HTML5 Editor, Notepad++, or any other HTML5 editor
Frontend	HTML5, CSS3, JavaScript, jQuery, Bootstrap/Angular/ReactJS, XML, and Figma UI Toolkit
AI tools	Figma AI, Canva Magic, Framer AI, Uizard, Visily AI, Penpot, GitHub Copilot, Tabnine, Windsurf, and ChatGPT, Claude, or Gemini for UI design, code assistance, image generation, and FAQ/content generation. Chatbot Platforms such as Tawk.to and Tidio for chatbot integration
Data Store	JSON files or TXT files

1.9 Project Deliverables

You have to design and build the project and submit it along with a complete project report that includes:

- Problem Definition
- Design specifications
- Diagrams such as flowcharts for various activities, Data Flow Diagrams, and so on
- Test Data Used in the Project
- Project Installation Instructions (MANDATORY)

Documentation is considered a very important part of the project. Ensure that documentation is complete and comprehensive. Documentation should not contain any source code.

The consolidated project will be submitted as a zip file with a ReadMe.doc file listing assumptions (if any) made at your end and JSON/TXT files containing test data.

Over and above the given specifications, you can apply your creativity and logic to improve the Website.

Optionally, a live hosted URL can be supplied for the site.

Submit a video (mp4 file) demonstrating the working of the Website including all the functionalities of the project. This is MANDATORY.

~~~ End of Document ~~~